INDEX

Sl no.	Name and Topics	Page no.
Chapter 1	**INTRODUCTION AND OBJECTIVE**	**1-3**
	1.1 Introduction	
	1.2 Project Profile	
	1.3 Objectives	
	1.4 Existing Manual System	
	1.4.1 Limitations of Existing Manual System	
	1.5 Proposed System	
	1.5.1 Advantages of Proposed System	
Chapter 2	**REQUIREMENT ANALYSIS**	**4-6**
	2.1 Introduction	
	2.2 Software Requirements	
	2.3 Hardware Requirements	
	2.4 Training Requirement	
	2.5 Description	
	2.6 Why C++?	
	2.7 What is VS Code?	
	2.8 What is MinGW?	
Chapter 3	**FEASIBILITY STUDY**	**7-8**
	3.1 Introduction	
	3.2 Technical Feasibility	
	3.3 Economic Feasibility	
	3.4 Operational Feasibility	
	3.5 Conclusion	

INDEX

Sl no.	Name and Topics	Page no.
Chapter 4	**CODING AND OUTPUT**	**10-29**
	4.1 Coding	
	4.2 Output	
Chapter 5	**CONCLUSION**	**30-31**
	5.1 Future Scope of the Project	
	5.2 Limitations	
	5.3 Security	
	5.4 Conclusion	
Chapter 6	**BIBLIOGRAPHY**	**32**
	6.1 Books	

CHAPTER 1: INTRODUCTION AND OBJECTIVE

These manual methods are often time-consuming, prone to human error, and inefficient during peak usage.

1.1 Introduction:

In today's world, technology plays a crucial role in simplifying daily tasks and improving efficiency. Educational institutions, including schools and colleges, have increasingly adopted computerized systems to manage their operations smoothly. Traditional library systems rely on manual registers for recording book usage, student details, staff information, and daily activities.

The proposed "Library Management System" is a computerized solution developed using C++ programming. It automates essential library operations such as student registration, login authentication, staff management, book viewing, book selection, and history tracking. The system incorporates file handling techniques to store and retrieve data securely, ensuring that library processes become more organized and easy to manage.

With increasing academic workloads and high demand for automation in educational institutions, this project addresses the limitations of manual record-keeping. It provides a reliable and user-friendly interface for students and administrators to perform daily tasks efficiently. The system aims to reduce errors, save time, and enhance the overall productivity of library operations.

1.2 Project Profile:

- **Project Title:** Library Management System
- **Platform:** Windows or Linux
- **Project Type:** Application Software
- **Category:** File Handling Based System
- **Language:** C++

1.3 Objectives:

The Library Management System is designed to achieve the following objectives for efficient and simplified library operations:

- To streamline student registration and login processes using secure ID, name, and password verification through file handling.
- To maintain accurate staff and timing records and allow the admin to add, edit, or remove entries easily.
- To automate the storage and retrieval of user data, reducing dependency on manual registers.
- To ensure secure handling of book-count data, preventing errors in tracking the number of readers for each book.
- To maintain a history of unregistered students for administrative accuracy and reference.
- To offer a simple, user-friendly interface so even non-technical users can operate the system effortlessly.

1.4 Existing Manual System:

The existing manual library system involves maintaining handwritten registers for recording student details, book usage, staff information, and staff timings. This process includes noting down names, book choices, and visit details in paper-based logbooks. While this method is simple, it is prone to numerous challenges such as errors in entries, difficulty in searching old records, and risk of data loss.

1.4.1 Limitations of the Existing Manual System:

- **Time-consuming:** Manual recording of every transaction slows down daily operations.
- **Higher chances of errors:** Mistakes such as wrong entries, missing records, or miscalculations are common.
- **Difficult to retrieve information:** Searching through registers takes time and can delay work.
- **Risk of damage or loss:** Physical records can be torn, misplaced, or lost permanently.
- **Not suitable for large data:** Handling a growing number of books and users becomes unmanageable.

1.5 Proposed System:

The proposed system is a computer-based solution designed to overcome the drawbacks of manual methods. It automates major processes such as order management, billing, and data storage. By using C++ file handling, the system provides faster operations, accurate record-keeping, and improved security. This modern approach ensures smoother workflow, quick data access, and better management of customer information.

1.5.1 Advantages of Proposed System:

- **Operational Efficiency:** Reduces repetitive manual work by automating core library tasks, saving both time and effort.
- **Accuracy:** Ensures precise record-keeping and minimizes mistakes in book transactions and user data.
- **Security:** Provides safer data storage through structured file handling, reducing risks of losing or tampering records.
- **Scalability:** Can be expanded easily to include more books, more users, and additional features in the future.
- **User Convenience:** Improves the overall experience for librarians and students by offering faster access to information and smoother operations.
- **Better Organization:** Keeps all book and user records neatly arranged, reducing confusion and mismanagement.

CHAPTER 2: REQUIREMENT ANALYSIS

2.1 Introduction:

The Library Management System in this project is developed using the C++ programming language. This system helps make library work easier by automating tasks such as registering students, managing books, checking available books and storing details about users through file handling.

The program uses C++ file handling to store user information, book counts, and unregistration history. This makes the system simple, fast, and easy to manage. By reducing manual work, the system lowers the chances of error and provides a better way to organize library data. It is designed to be user-friendly so that both students and admins can use it without difficulty.

2.2 Software Requirements:

- ✓ Visual Studio Code
- ✓ MinGW (Minimalist GNU for Windows)
- ✓ Windows or Linux Operating System

2.3 Hardware Requirements:

- ✓ A computer with at least 2GB RAM and a 1.5GHz processor (minimum requirement)
- ✓ Adequate storage space for program files and data records

2.4 Training Requirement:

Basic training is necessary to effectively use the Library Management System. Users should have:

- A basic understanding of C++ programming, allowing them to recognize how the system operates.
- Knowledge of file handling concepts, as the system depends on reading and writing data efficiently.
- Familiarity with general library operations, enabling smooth integration of the system into daily tasks.

Meeting these training requirements ensures that users can utilise all features of the system and maintain accurate and well-structured library records.

2.5 Description:

The Library Management System helps manage important library activities that are needed for the smooth functioning of a library. The system supports student registration, book selection, book availability checking, and viewing library rules. For administrators, the system provides features like updating timings, and managing book information. By using this program, daily tasks become faster and more organized.

For students, the system stores important details such as name, ID number, address, contact information, and selected books. This information is saved safely using file handling. The book menu allows students to browse different books available in the library and select the ones they want.

For administrators, all staff details, book records, and timings are stored in files, making the system reliable and easy to update. These features help improve library operations by reducing manual effort and saving time.

C++ plays an important role in building the Library Management System because it provides fast performance and strong file-handling abilities. As an object-oriented programming language, C++ helps organize the system into different classes such as Student, and Books, making the program easier to maintain and update.

The file-handling feature of C++ is especially useful because it allows the program to store and retrieve student records, staff details, and book lists safely without losing data. For example, C++ file functions help save and load book information and student details with very little risk of corruption. C++ also performs tasks quickly, which helps the system manage operations like book selection, registration, and updating records without delays. This improves the overall efficiency of the library, reduces errors, and makes daily work easier for both students and staff.

2.6 Why C++?

C++ is chosen for the Library Management System because of its speed, reliability, and strong support for file handling. It is simple to use for building menu-driven programs and works well even on computers with low hardware capability. Below are three important reasons why C++ is suitable for this project:

1. **Efficiency:** C++ is a fast programming language that quickly executes tasks, which is important when handling multiple library functions at the same time.
2. **File Handling Capabilities:** C++ provides strong tools for storing and retrieving data from files. This is helpful for saving student, staff, and book records securely.
3. **Object-Oriented Programming (OOP):** OOP features help create modular and reusable code. This makes the project easier to develop, understand, and maintain.

2.7 What is VS Code?

Visual Studio Code (VS Code) is a free and lightweight code editor made by Microsoft. It is used for writing and editing programs in many languages like C++, Python, Java, and more. VS Code provides helpful features such as syntax highlighting, auto-suggestions, debugging tools, and extensions that make coding easier and faster. It is simple to use and widely preferred by students and developers for programming.

2.8 What is MinGW?

MinGW (Minimalist GNU for Windows) is a tool that provides the GCC compiler for Windows. It allows you to compile and run C and C++ programs on your computer. In simple words, MinGW converts the code you write into an executable program. It is lightweight, easy to install, and commonly used with editors like VS Code for running C++ programs.

CHAPTER 3: FEASIBILITY STUDY

3.1 Introduction:

A feasibility study checks whether the project is practical, useful, and suitable for real use. It helps determine if the system can be created with the resources, technology, and skills available. This study also identifies challenges that may come during development or implementation. By examining these factors, it becomes clear whether the Library Management System can achieve its goals within the given conditions.

This chapter explains the technical, operational, and financial aspects of the system, giving a complete assessment of its readiness for use in a real library environment.

3.2 Technical Feasibility:

Technical feasibility evaluates whether the system can be developed with the available technology and skills. The Library Management System is built using C++, which is efficient, fast, and easy to implement for school-level projects. C++ provides strong support for:

- File handling (to save and view registered users)
- Arrays and classes (to store books, staff data, and user records)
- Simple input–output operations (making the system user-friendly)

The program includes menus for both students and administrators, making navigation easy even for users with limited technical knowledge. Because the system uses basic C++ features and does not require advanced hardware, it is completely feasible to implement on normal computers used in schools. The design is also flexible enough to add more features in the future, such as search options or book categories.

3.3 Economic Feasibility:

Economic feasibility examines whether the system is cost-effective. Since the Library Management System is developed using free tools like C++ and requires no additional software purchases, the cost of implementation is extremely low. The system reduces the need for manual record-keeping by storing user data in a file, which saves time for staff and reduces paperwork.

Because the project uses existing school computers and does not require new devices, it is a budget-friendly solution. Its simple design also lowers training costs because both students and staff can learn to use it quickly. Overall, the system is financially feasible and suitable for educational institutions with limited budgets.

3.4 Operational Feasibility:

Operational feasibility checks whether the system can smoothly fit into the daily working of the library. The Library Management System supports tasks such as:

- Displaying library rules
- Showing available books
- Registering students for reading
- Allowing admins to manage records
- Maintaining staff, timing, and cleaning-staff information.

The program is simple, interacts directly with the user, and provides clear instructions. Even users with little computer experience can use it easily. It reduces manual effort, prevents errors in record-keeping, and ensures that book limits (5 users per book per day) are properly followed. Therefore, the system is operationally practical and ready to be used in a real library environment.

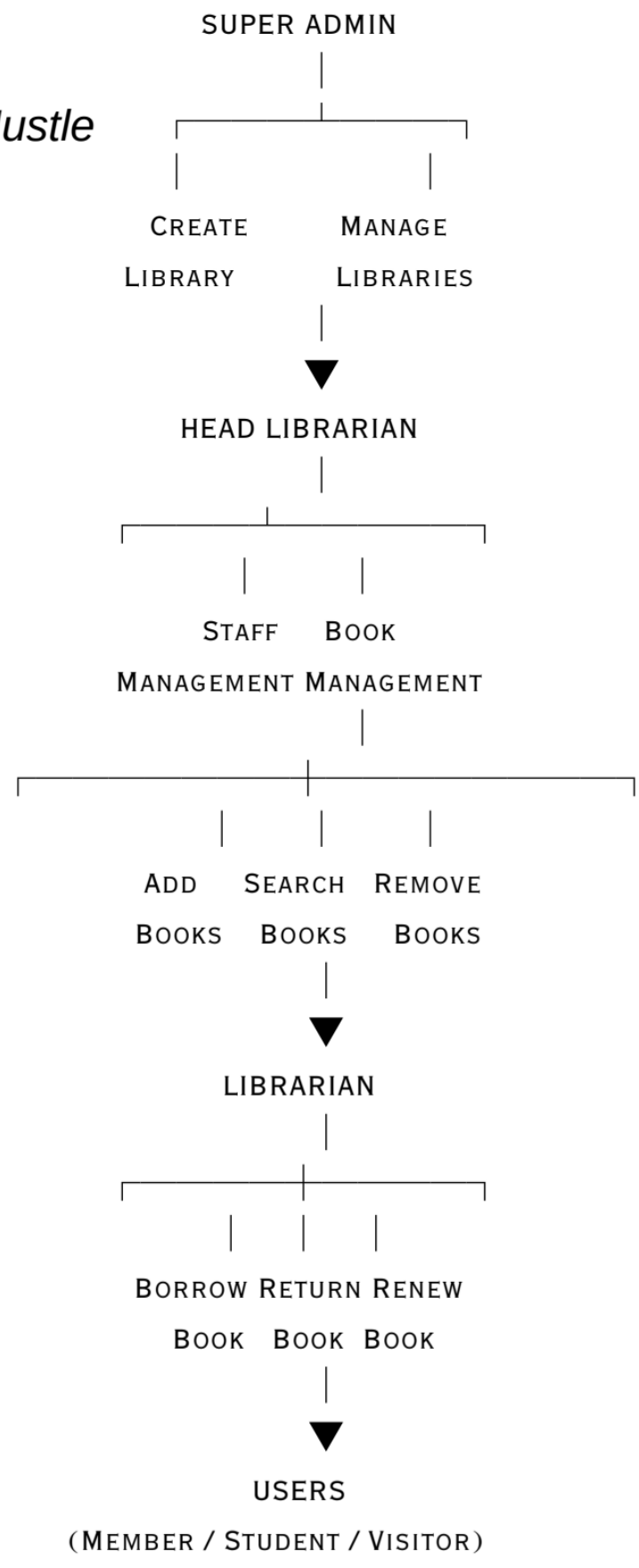
3.5 Conclusion:

The feasibility analysis shows that the Library Management System is a reliable and cost-effective solution for modern library management. By automating major tasks, reducing manual effort, and ensuring accuracy, the system lowers operational costs and improves productivity. Its scalable and simple design supports long-term use and reduces future maintenance needs. Overall, the system is economically and operationally suitable for educational institutions that want an efficient and user-friendly library management solution.

BLUEPRINT :

Library Management System

By Arrays Hustle



CHAPTER 4: CODING AND OUTPUT

4.1 Coding:

The main functionalities implemented in the Library Management System include:

- **Role-based Login:** The main menu lets the user enter as Super Admin, Librarian or Visitor. Super Admin and Librarian access is protected by a password.
- **Add Book:** Adds a new book with its title and author; a unique book ID is generated automatically.
- **List Books:** Displays all books along with their availability status (Available / Issued).
- **Search Book:** Finds books by a keyword from the title or the author's name.
- **Add Member and List Members:** Registers library members and shows how many books each member has issued.
- **Issue Book:** Issues an available book to a member; a member can hold at most 3 books at a time.
- **Return Book:** Returns an issued book after checking that the member ID matches the issue record.
- **Report:** Shows the total, available and issued books and the total number of members.
- **File Handling:** Book and member records are saved in library_books.txt and library_members.txt so that data is kept after the program closes.
- **Object-Oriented Design:** An abstract User class with SuperAdmin, Librarian and Visitor classes derived from it.

Source code:

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <cstdlib>
using namespace std;

struct Book {
    int id;
    string title, author;
    bool issued;
    int memberId;
```

```
};

struct Member {
    int id;
    string name;
    int booksIssued;
};

class User {
protected:
    string name;
public:
    User(string n = "") : name(n) {}
    virtual void panel() = 0;
    virtual ~User() {}
};

class SuperAdmin : public User {
public:
    SuperAdmin() : User("Super Admin") {}
    void panel() override { cout << "Logged in as Super Admin\n"; }
};

class Librarian : public User {
public:
    Librarian() : User("Librarian") {}
    void panel() override { cout << "Logged in as Librarian\n"; }
};

class Visitor : public User {
public:
    Visitor() : User("Visitor") {}
```

```
    void panel() override { cout << "Logged in as Visitor\n"; }
};

class Library {
    vector<Book> books;
    vector<Member> members;
    const string bookFile = "library_books.txt";
    const string memberFile = "library_members.txt";

    int findBook(int id) {
        for (int i = 0; i < (int)books.size(); i++)
            if (books[i].id == id) return i;
        return -1;
    }

    int findMember(int id) {
        for (int i = 0; i < (int)members.size(); i++)
            if (members[i].id == id) return i;
        return -1;
    }

    void save() {
        ofstream bf(bookFile), mf(memberFile);
        if (!bf || !mf) {
            cout << "Error: Unable to open data files.\n";
            return;
        }

        for (const Book &b : books)
            bf << b.id << '|' << b.title << '|' << b.author << '|'
               << b.issued << '|' << b.memberId << '\n';

        for (const Member &m : members)
```

```
        mf << m.id << '|' << m.name << '|' << m.booksIssued << '\n';
}

bool splitBook(const string &line, Book &b) {
    size_t p1 = line.find('|');
    size_t p2 = line.find('|', p1 == string::npos ? 0 : p1 + 1);
    size_t p3 = line.find('|', p2 == string::npos ? 0 : p2 + 1);
    size_t p4 = line.find('|', p3 == string::npos ? 0 : p3 + 1);

    if (p1 == string::npos || p2 == string::npos ||
        p3 == string::npos || p4 == string::npos) return false;

    try {
        b.id = stoi(line.substr(0, p1));
        b.title = line.substr(p1 + 1, p2 - p1 - 1);
        b.author = line.substr(p2 + 1, p3 - p2 - 1);
        b.issued = stoi(line.substr(p3 + 1, p4 - p3 - 1)) != 0;
        b.memberId = stoi(line.substr(p4 + 1));
    }
    catch (...) {
        return false;
    }
    return true;
}

bool splitMember(const string &line, Member &m) {
    size_t p1 = line.find('|');
    size_t p2 = line.find('|', p1 == string::npos ? 0 : p1 + 1);
    if (p1 == string::npos || p2 == string::npos) return false;

    try {
        m.id = stoi(line.substr(0, p1));
```

```
        m.name = line.substr(p1 + 1, p2 - p1 - 1);
        m.booksIssued = stoi(line.substr(p2 + 1));
    }
    catch (...) {
        return false;
    }
    return true;
}

void load() {
    ifstream bf(bookFile), mf(memberFile);
    string line;

    while (getline(bf, line)) {
        Book b;
        if (splitBook(line, b)) books.push_back(b);
    }

    while (getline(mf, line)) {
        Member m;
        if (splitMember(line, m)) members.push_back(m);
    }

    if (books.empty()) {
        books.push_back({1001, "The Alchemist", "Paulo Coelho", false, 0});
        books.push_back({1002, "Wings of Fire", "A.P.J. Abdul Kalam", false, 0});
        books.push_back({1003, "Harry Potter", "J.K. Rowling", false, 0});
        books.push_back({1004, "The Hobbit", "J.R.R. Tolkien", false, 0});
    }

    if (members.empty()) {
        members.push_back({501, "Rahul", 0});
```

```
        members.push_back({502, "Ananya", 0});
    }

    save();
}

void addBook() {
    Book b;
    b.id = books.empty() ? 1001 : books.back().id + 1;
    b.issued = false;
    b.memberId = 0;

    cout << "Enter title: ";
    getline(cin >> ws, b.title);
    cout << "Enter author: ";
    getline(cin, b.author);

    books.push_back(b);
    save();
    cout << "Book added successfully.\n";
}

void listBooks() {
    cout << "\n--- BOOKS ---\n";
    for (const Book &b : books)
        cout << b.id << " | " << b.title << " | " << b.author
             << " | " << (b.issued ? "Issued" : "Available") << '\n';
}

void searchBook() {
    string key;
    cout << "Enter title/author keyword: ";
    getline(cin >> ws, key);
```

```
    bool found = false;
    for (const Book &b : books) {
        if (b.title.find(key) != string::npos ||
            b.author.find(key) != string::npos) {
            cout << b.id << " | " << b.title << " | " << b.author
                 << " | " << (b.issued ? "Issued" : "Available") << '\n';
            found = true;
        }
    }
    if (!found) cout << "No matching book found.\n";
}

void addMember() {
    Member m;
    m.id = members.empty() ? 501 : members.back().id + 1;
    m.booksIssued = 0;

    cout << "Enter member name: ";
    getline(cin >> ws, m.name);

    members.push_back(m);
    save();
    cout << "Member added. ID: " << m.id << '\n';
}

void listMembers() {
    cout << "\n--- MEMBERS ---\n";
    for (const Member &m : members)
        cout << m.id << " | " << m.name
             << " | Books issued: " << m.booksIssued << '\n';
}
```

```
bool readNumber(const string &message, int &value) {
    cout << message;
    if (cin >> value) return true;
    cin.clear();
    cin.ignore(1000, '\n');
    cout << "Invalid input.\n";
    return false;
}

void issueBook() {
    int bid, mid;
    if (!readNumber("Enter book ID: ", bid)) return;
    if (!readNumber("Enter member ID: ", mid)) return;

    int b = findBook(bid), m = findMember(mid);
    if (b == -1 || m == -1) {
        cout << "Invalid book or member ID.\n";
        return;
    }
    if (books[b].issued) {
        cout << "Book is already issued.\n";
        return;
    }
    if (members[m].booksIssued >= 3) {
        cout << "Member has reached the 3-book limit.\n";
        return;
    }

    books[b].issued = true;
    books[b].memberId = mid;
    members[m].booksIssued++;
```

```
    save();
    cout << "Book issued successfully.\n";
}

void returnBook() {
    int bid, mid;
    if (!readNumber("Enter book ID: ", bid)) return;

    int b = findBook(bid);
    if (b == -1) {
        cout << "Book not found.\n";
        return;
    }
    if (!books[b].issued) {
        cout << "Book is not currently issued.\n";
        return;
    }

    if (!readNumber("Enter member ID of the person returning the book: ", mid))
        return;

    if (mid != books[b].memberId) {
        cout << "Member ID does not match the issued record.\n";
        return;
    }

    int m = findMember(mid);
    if (m != -1 && members[m].booksIssued > 0)
        members[m].booksIssued--;

    books[b].issued = false;
    books[b].memberId = 0;
    save();
```

```
    cout << "Book returned successfully.\n";
}

void report() {
    int available = 0, issued = 0;
    for (const Book &b : books)
        b.issued ? issued++ : available++;

    cout << "\n--- REPORT ---\n";
    cout << "Total books: " << books.size() << '\n';
    cout << "Available: " << available << '\n';
    cout << "Issued: " << issued << '\n';
    cout << "Total members: " << members.size() << '\n';
}

void pause() {
    cout << "\nPress Enter to continue...";
    cin.ignore(1000, '\n');
    cin.get();
}

void adminMenu() {
    int c;
    do {
        system("cls");
        cout << "===== SUPER ADMIN PANEL =====\n";
        cout << "1. Add Book\n2. List Books\n3. Search Book\n";
        cout << "4. Add Member\n5. List Members\n6. Issue Book\n";
        cout << "7. Return Book\n8. Report\n9. Logout\nChoice: ";

        if (!(cin >> c)) {
            cin.clear();
```

```
            cin.ignore(1000, '\n');
            c = 0;
        }

        switch (c) {
            case 1: addBook(); break;
            case 2: listBooks(); break;
            case 3: searchBook(); break;
            case 4: addMember(); break;
            case 5: listMembers(); break;
            case 6: issueBook(); break;
            case 7: returnBook(); break;
            case 8: report(); break;
            case 9: break;
            default: cout << "Invalid choice.\n";
        }

        if (c != 9) pause();
    } while (c != 9);
}

void librarianMenu() {
    int c;
    do {
        system("cls");
        cout << "===== LIBRARIAN PANEL =====\n";
        cout << "1. Add Book\n2. List Books\n3. Search Book\n";
        cout << "4. List Members\n5. Issue Book\n6. Return Book\n7. Logout\nChoice: ";

        if (!(cin >> c)) {
            cin.clear();
            cin.ignore(1000, '\n');
```

```
            c = 0;
        }

        switch (c) {
            case 1: addBook(); break;
            case 2: listBooks(); break;
            case 3: searchBook(); break;
            case 4: listMembers(); break;
            case 5: issueBook(); break;
            case 6: returnBook(); break;
            case 7: break;
            default: cout << "Invalid choice.\n";
        }

        if (c != 7) pause();
    } while (c != 7);
}

void visitorMenu() {
    int c;
    do {
        system("cls");
        cout << "===== VISITOR PANEL =====\n";
        cout << "1. List Available Books\n2. Search Book\n";
        cout << "3. Exit\nChoice: ";

        if (!(cin >> c)) {
            cin.clear();
            cin.ignore(1000, '\n');
            c = 0;
        }
```

```
        switch (c) {
            case 1:
                cout << "\n--- AVAILABLE BOOKS ---\n";
                for (const Book &b : books)
                    if (!b.issued)
                        cout << b.id << " | " << b.title
                             << " | " << b.author << '\n';
                break;
            case 2: searchBook(); break;
            case 3: break;
            default: cout << "Invalid choice.\n";
        }

        if (c != 3) pause();
    } while (c != 3);
}

void login() {
    int role, pass;

    do {
        system("cls");
        cout << "===== LIBRARY MANAGEMENT SYSTEM =====\n";
        cout << "1. Super Admin\n2. Librarian\n3. Visitor\n4. Exit\nChoice: ";

        if (!(cin >> role)) {
            cin.clear();
            cin.ignore(1000, '\n');
            role = 0;
        }

        User *user = nullptr;
```

```
if (role == 1) {
    if (!readNumber("Password: ", pass)) continue;
    if (pass != 123) {
        cout << "Wrong password.\n";
        pause();
        continue;
    }
    user = new SuperAdmin();
    user->panel();
    delete user;
    adminMenu();
}
else if (role == 2) {
    if (!readNumber("Password: ", pass)) continue;
    if (pass != 456) {
        cout << "Wrong password.\n";
        pause();
        continue;
    }
    user = new Librarian();
    user->panel();
    delete user;
    librarianMenu();
}
else if (role == 3) {
    user = new Visitor();
    user->panel();
    delete user;
    visitorMenu();
}
```

```
            else if (role != 4) {
                cout << "Invalid choice.\n";
                pause();
            }
        } while (role != 4);
    }

public:
    Library() { load(); }
    ~Library() { save(); }

    void run() { login(); }
};

int main() {
    Library library;
    library.run();
    return 0;
}
```

4.2 Output:

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Super Admin
2. Librarian
3. Visitor
4. Exit
Choice:
```

Main menu

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Super Admin
2. Librarian
3. Visitor
4. Exit
Choice: 1
Password: 123
Logged in as Super Admin
```

Super admin login

```
===== SUPER ADMIN PANEL =====
1. Add Book
2. List Books
3. Search Book
4. Add Member
5. List Members
6. Issue Book
7. Return Book
8. Report
9. Logout
Choice:
```

Super admin panel

```
===== SUPER ADMIN PANEL =====
1. Add Book
2. List Books
3. Search Book
4. Add Member
5. List Members
6. Issue Book
7. Return Book
8. Report
9. Logout
Choice: 1
Enter title: The Monk Who Sold His Ferrari
Enter author: Robin Sharma
Book added successfully.

Press Enter to continue...
```

Add book

```
===== SUPER ADMIN PANEL =====
1. Add Book
2. List Books
3. Search Book
4. Add Member
5. List Members
6. Issue Book
7. Return Book
8. Report
9. Logout
Choice: 2

--- BOOKS ---
1001 | The Alchemist | Paulo Coelho | Available
1002 | Wings of Fire | A.P.J. Abdul Kalam | Available
1003 | Harry Potter | J.K. Rowling | Available
1004 | The Hobbit | J.R.R. Tolkien | Available
1005 | The Monk Who Sold His Ferrari | Robin Sharma | Available

Press Enter to continue...
```

List books

```
===== SUPER ADMIN PANEL =====
1. Add Book
2. List Books
3. Search Book
4. Add Member
5. List Members
6. Issue Book
7. Return Book
8. Report
9. Logout
Choice: 3
Enter title/author keyword: Harry
1003 | Harry Potter | J.K. Rowling | Available

Press Enter to continue...
```

Search book

```
===== SUPER ADMIN PANEL =====
1. Add Book
2. List Books
3. Search Book
4. Add Member
5. List Members
6. Issue Book
7. Return Book
8. Report
9. Logout
Choice: 4
Enter member name: Kalpita
Member added. ID: 503

Press Enter to continue...
```

Add member

```
===== SUPER ADMIN PANEL =====
1. Add Book
2. List Books
3. Search Book
4. Add Member
5. List Members
6. Issue Book
7. Return Book
8. Report
9. Logout
Choice: 5

--- MEMBERS ---
501 | Rahul | Books issued: 0
502 | Ananya | Books issued: 0
503 | Kalpita | Books issued: 0

Press Enter to continue...
```

List members

```
===== SUPER ADMIN PANEL =====
1. Add Book
2. List Books
3. Search Book
4. Add Member
5. List Members
6. Issue Book
7. Return Book
8. Report
9. Logout
Choice: 6
Enter book ID: 1001
Enter member ID: 501
Book issued successfully.

Press Enter to continue...
```

Issue book

```
===== SUPER ADMIN PANEL =====
1. Add Book
2. List Books
3. Search Book
4. Add Member
5. List Members
6. Issue Book
7. Return Book
8. Report
9. Logout
Choice: 6
Enter book ID: 1001
Enter member ID: 502
Book is already issued.

Press Enter to continue...
```

Book already issued

```
===== SUPER ADMIN PANEL =====
1. Add Book
2. List Books
3. Search Book
4. Add Member
5. List Members
6. Issue Book
7. Return Book
8. Report
9. Logout
Choice: 2

--- BOOKS ---
1001 | The Alchemist | Paulo Coelho | Issued
1002 | Wings of Fire | A.P.J. Abdul Kalam | Available
1003 | Harry Potter | J.K. Rowling | Available
1004 | The Hobbit | J.R.R. Tolkien | Available
1005 | The Monk Who Sold His Ferrari | Robin Sharma | Available

Press Enter to continue...
```

Books after issuing

```
===== SUPER ADMIN PANEL =====
1. Add Book
2. List Books
3. Search Book
4. Add Member
5. List Members
6. Issue Book
7. Return Book
8. Report
9. Logout
Choice: 7
Enter book ID: 1001
Enter member ID of the person returning the book: 501
Book returned successfully.

Press Enter to continue...
```

Return book

```
===== SUPER ADMIN PANEL =====
1. Add Book
2. List Books
3. Search Book
4. Add Member
5. List Members
6. Issue Book
7. Return Book
8. Report
9. Logout
Choice: 8

--- REPORT ---
Total books: 5
Available: 5
Issued: 0
Total members: 3

Press Enter to continue...
```

Library report

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Super Admin
2. Librarian
3. Visitor
4. Exit
Choice: 1
Password: 999
Wrong password.

Press Enter to continue...
```

Wrong password

```
===== LIBRARY MANAGEMENT SYSTEM =====
1. Super Admin
2. Librarian
3. Visitor
4. Exit
Choice: 2
Password: 456
Logged in as Librarian
```

Librarian login

```
===== LIBRARIAN PANEL =====
1. Add Book
2. List Books
3. Search Book
4. List Members
5. Issue Book
6. Return Book
7. Logout
Choice: 5
Enter book ID: 1004
Enter member ID: 502
Book issued successfully.

Press Enter to continue...
```

Librarian issues a book

```
===== VISITOR PANEL =====
1. List Available Books
2. Search Book
3. Issue Book
4. Return Book
5. Exit
Choice: 1

--- AVAILABLE BOOKS ---
1001 | The Alchemist | Paulo Coelho
1002 | Wings of Fire | A.P.J. Abdul Kalam
1003 | Harry Potter | J.K. Rowling
1004 | The Hobbit | J.R.R. Tolkien

Press Enter to continue...
```

Visitor panel

```
===== VISITOR PANEL =====
1. List Available Books
2. Search Book
3. Issue Book
4. Return Book
5. Exit
Choice: 4
Enter book ID: 1002
Enter member ID of the person returning the book: 502
Member ID does not match the issued record.

Press Enter to continue...
```

Wrong member ID on return

CHAPTER 5: CONCLUSION

5.1 Future Scope of the Project:

- ❖ Add a graphical user interface (GUI) using frameworks like Qt, Tkinter, or JavaFX to make the Library Management System easier to use and more attractive.
- ❖ Introduce database support instead of text files to store student records, book details, staff lists, and history more safely and efficiently.
- ❖ Add features such as book borrowing, return dates, fine calculation, and automatic notifications.
- ❖ Improve system security by adding encrypted file storage, password hashing, and safer login verification.
- ❖ Add support for multiple admins, network-based access, or cloud backup to make the system usable in bigger institutions.

5.2 Limitations:

- The system works through a command-line interface, which may not be comfortable for all users.
- It uses basic text files for storing information, which limits performance and is less secure than a database.
- The system does not include modern encryption methods to protect user data and book records.
- It is not designed for multi-user access or network environments, so multiple students cannot use it at the same time.
- The system mainly manages registration, book selection, and basic staff/worker records, but does not include advanced library features like book loans or search filters.

5.3 Security:

The system keeps data by saving records in simple text files. This allows easy access and editing but does not provide strong protection. To improve safety, the system can use advanced encryption techniques such as AES or RSA to protect sensitive information like passwords and student IDs. Using hashing for passwords and secure storage methods would prevent unauthorized changes and protect user privacy. These improvements would bring the system closer to modern security standards used in digital library applications.

5.4 Conclusion:

The Library Management System is a helpful tool for organizing and managing daily library activities. It reduces manual effort by automating important tasks such as student registration, book selection, viewing staff and worker information, and maintaining user history. The system also shows how C++ can be used effectively for file handling and simple record management.

Its design can be improved and expanded in the future with features such as borrowing and returning books, better security, and graphical interfaces. The system provides a strong foundation for learning and helps in understanding how real library operations can be handled using computer applications.

Overall, the Library Management System is simple, efficient, and useful. It meets the basic needs of a modern library and can be further developed into a more advanced and complete library solution.

CHAPTER 6: BIBLIOGRAPHY

6.1 Books:

- "Object-Oriented Programming with C++" by E. Balagurusamy
- "The C++ Programming Language" by Bjarne Stroustrup
- "C++ Primer" by Stanley B. Lippman, Josée Lajoie, and Barbara E. Moo
- "Programming: Principles and Practice Using C++" by Bjarne Stroustrup
- "Effective Modern C++" by Scott Meyers
- "C++ Concurrency in Action" by Anthony Williams
- "Accelerated C++: Practical Programming by Example" by Andrew Koenig and Barbara E. Moo